SIMATS ENGINEERING

CO-3: **ASSESSMENT TOOL 1**

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

I _CH.NAGA SAI SRIKANTH (192311159) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

CH.NAGASAISRIKANTH

Signature of the student:

Annexure A
Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

ANSWERS

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

```python
import hashlib
import time

messages = ["Hello", "Hello World", "Cryptography"]

for msg in messages:
    print("\nMessage:", msg)

    start = time.time()
    md5 = hashlib.md5(msg.encode()).hexdigest()
    md5_time = time.time() - start

    start = time.time()
    sha = hashlib.sha256(msg.encode()).hexdigest()
    sha_time = time.time() - start

    print("MD5    :", md5)
    print("SHA-256:", sha)

    print("MD5 Time    :", md5_time)
    print("SHA-256 Time:", sha_time)
```

Console

```
Message:
 Hello
MD5    : 8b1a9953c4611296a827abf8c47804d7
SHA-256: 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
MD5 Time    : 0.0009999275207519531
SHA-256 Time: 0.0009999275207519531

Message:
 Hello World
MD5    : b10a8db164e0754105b7a99be72e3fe5
SHA-256: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
MD5 Time    : 0.0
SHA-256 Time: 0.0

Message:
 Cryptography
MD5    : 64ef07ce3e4b420c334227eecb3b3f4c
SHA-256: b584eec728548aced5a66c0267dd520a00871b5e7b735b2d8202f86719f61857
MD5 Time    : 0.0
SHA-256 Time: 0.0
```

2.

Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

```python
import hashlib

p = 61
q = 53

n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = pow(e, -1, phi)

print("Public Key :", (e, n))
print("Private Key:", (d, n))

message = input("Enter message: ")

h = int(hashlib.sha256(message.encode()).hexdigest(), 16)

signature = pow(h, d, n)

print("Signature:", signature)

h2 = int(hashlib.sha256(message.encode()).hexdigest(), 16)

check = pow(signature, e, n)

if check == h2 % n:
    print("Signature is VALID")
else:
    print("Signature is INVALID")
```

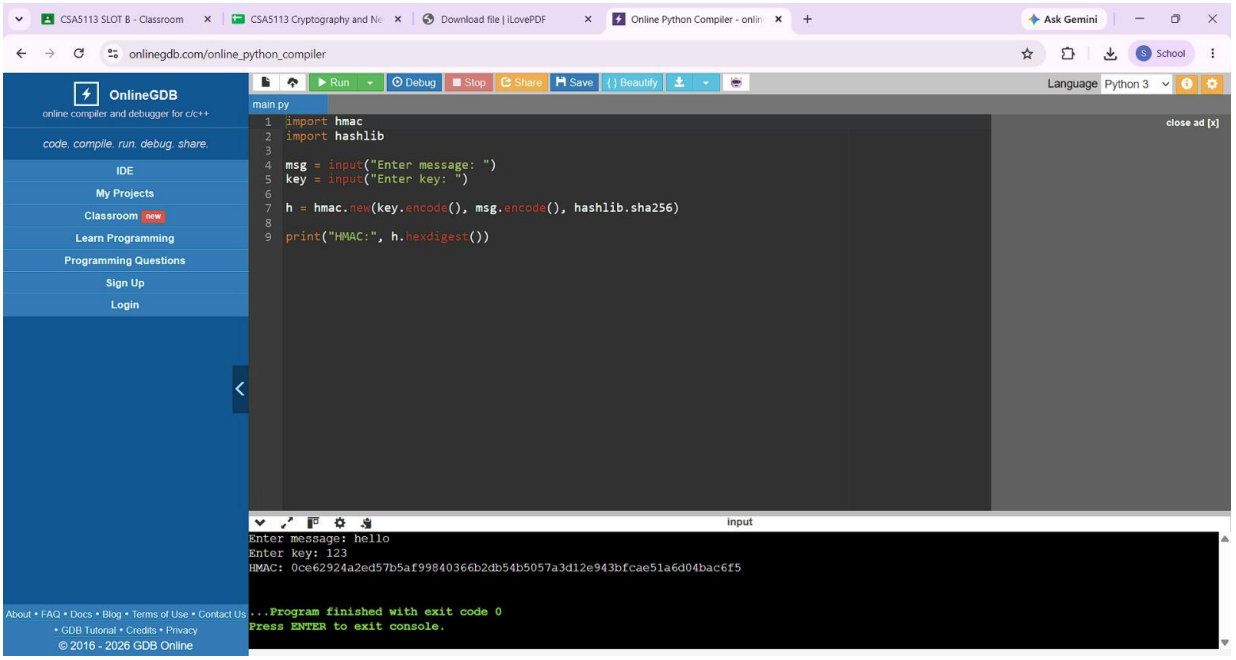
Input

```
Enter message: Hello
```

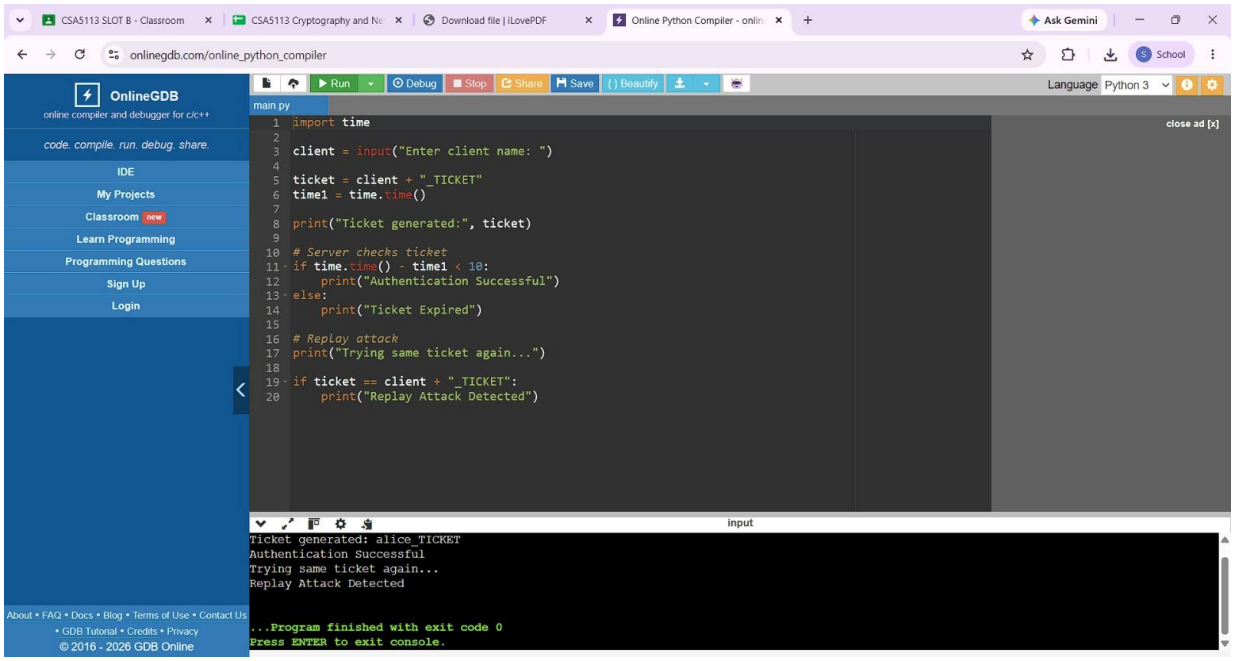
Output

```
Public Key : (17, 3233)
Private Key: (2753, 3233)
Enter message: Hello
Signature: 2460
Signature is VALID
```

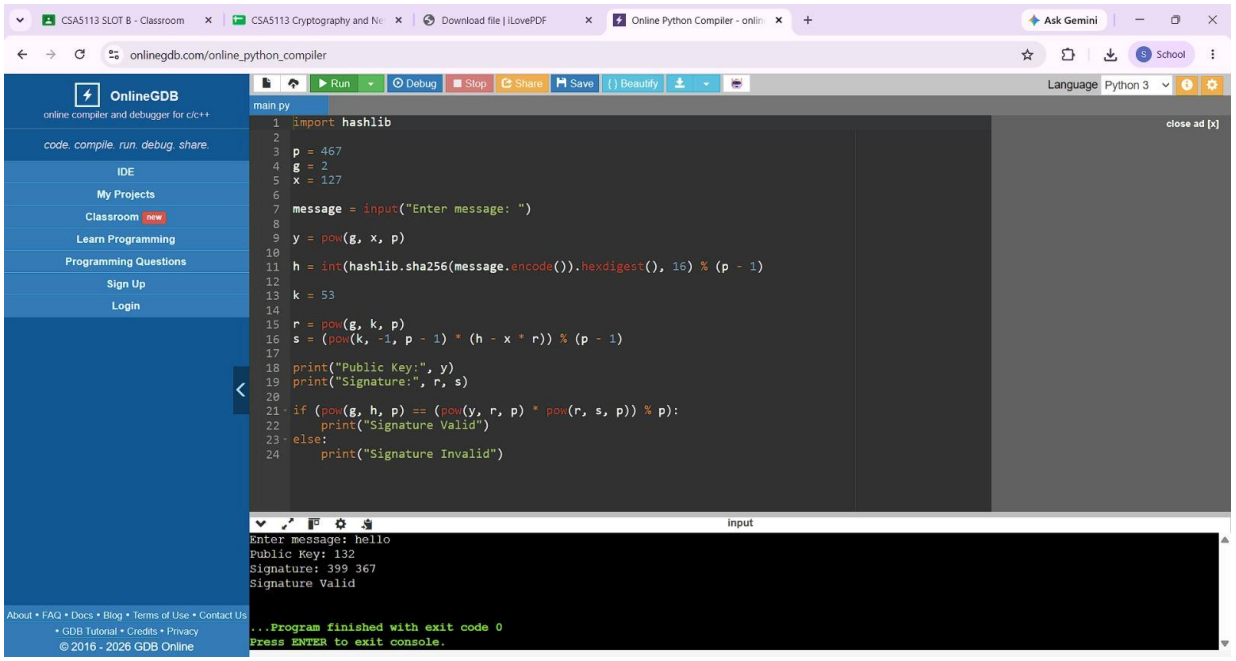
3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.



4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.



5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.


CSA5113 SLOT B - Classroom
CSA5113 Cryptography and Ne
Download file | iLovePDF
Online Python Compiler - onlin
Ask Gemini
onlinegdb.com/online_python_compiler
School
OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.
IDE
My Projects
Classroom new
Learn Programming
Programming Questions
Sign Up
Login
Run
Debug
Stop
Share
Save
{} Beautify
Language
Python 3
main.py
close ad [x]
import hashlib
p = 467
g = 2
x = 127
message = input("Enter message: ")
y = pow(g, x, p)
h = int(hashlib.sha256(message.encode()).hexdigest(), 16) % (p - 1)
k = 53
r = pow(g, k, p)
s = (pow(k, -1, p - 1) * (h - x * r)) % (p - 1)
print("Public Key:", y)
print("Signature:", r, s)
if (pow(g, h, p) == (pow(y, r, p) * pow(r, s, p)) % p):
print("Signature Valid")
else:
print("Signature Invalid")
input
Enter message: hello
Public Key: 132
Signature: 399 367
Signature Valid
...Program finished with exit code 0
Press ENTER to exit console.
About • FAQ • Docs • Blog • Terms of Use • Contact Us
• GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementatio n with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation